AUDIT INDÉPENDANT · 12 OCTOBRE 2026

codex_test
Audit du fonctionnement local

Conclusion : le dépôt conserve une base de recherche substantielle et traçable. La dernière extension est reproductible à l'identique. En revanche, une installation neuve et une nouvelle reconstruction complète ne sont pas encore assez fiables pour être lancées sans précautions : dépendances omises, fuseau implicite, validation finale trop tôt et publication partielle des tables.

L'audit a été effectué par un seul agent, sans sous-agent. Il est calibré pour ton projet personnel local. Les corrections proposées restent dans le programme existant et visent les résultats, la conservation des données et la simplicité de relance.

Vérification	Résultat
Révision auditée	5c8d484cf8be8dafdfdf445844b77d456ff27c94
Suite sur les seules dépendances déclarées	24 échecs / 113 succès
Suite après ajout des dépendances absentes	137 succès
Rejeu de la dernière extension	56 observations identiques ; 8 tables identiques octet par octet
Documents bruts retrouvés par SHA-256	12 188 / 12 197 ; 9 métadonnées manquantes
Anomalie finale de qualification du graphe	2 arêtes monétaires vers une entité non confirmée

Ce que je corrigerais d'abord

1. Compléter requirements.txt. **2.** Fixer la convention de fuseau. **3.** Valider après toutes les extensions et résoudre les deux arêtes. **4.** Garder le dernier jeu de tables intact jusqu'à validation complète du suivant.

Le détail contient douze constats : quatre P1, cinq P2 et trois P3. P1 signifie « à corriger avant de se fier à une prochaine reconstruction » ; P2 concerne la reprise et la traçabilité ; P3 concerne la cohérence ou un cas local moins fréquent. Il ne s'agit pas d'une notation du risque financier des sociétés étudiées.

Aucune modification du code ou des données suivis du dépôt n'a été publiée ou conservée par cet audit. Les expérimentations ont utilisé des dossiers distincts.

PÉRIMÈTRE ET STRUCTURE

Ce que contient réellement le projet

Le dépôt est un pipeline Python de recherche financière : collecte de documents SEC et de sources officielles japonaises, extraction XBRL, observations rédigées et qualifiées, calculs DuckDB, graphes de relations, contrôles et export de huit tables Parquet. Le résultat courant porte sur vingt-six groupes principaux ; les identifiants externes et les paires produisent davantage de group_id dans certaines tables.

Étape	Fichiers principaux	Rôle audité
Acquisition	network.py, collect.py, archives.py, backup.py	Accès, cache, conservation et restauration
Faits et preuves	xbrl.py, mapping.py, observations.py, reader.py	Identité du fait, unité, période, citation et attribution
Calculs	calculation.py, quarter_series.py, ttm.py, tier1_numeric.py, numeric_extended.py	Sélection datée, périmètres, ratios et flux
Relations et événements	entities.py, graph.py, pairs.py, events.py, evaluation.py	Entités, financement, exclusions et conclusions
Extensions	softbank_integration.py, dell_integration.py, smci_integration.py, targeted_priorities.py	Conservation des états antérieurs et ajouts
Reprise et résultats	phase3.py, assembly_checkpoint.py, verify_delivery.py, reproduce.py, *_package.py	Rejeu, validation, exports et preuves

Inventaire figé : 7 826 fichiers suivis, 487 commits ; 89 modules secfragility, 11 406 lignes physiques ; 21 fichiers de tests, 1 435 lignes ; 2 738 scripts dans work/, souvent de petites sélections documentaires, pour 14 581 lignes. Les 2 848 fichiers Python suivis ont tous été analysés par le parseur Python sans erreur de syntaxe.

Données volumineuses : 169 objets Git LFS totalisent 5 584 177 224 octets, soit environ 5,20 Gio. Leurs empreintes finales ont toutes été vérifiées. Les données et sauvegardes pèsent bien davantage que le code : prévoir de l’espace pour le clone, les archives décompressées, les baselines et un nouveau calcul.

Couverture de l’audit : revue manuelle des chemins critiques ; analyse de syntaxe de tous les fichiers Python suivis ; tests ; requêtes globales sur les huit tables ; intégrité des sources conservées ; cinq reproductions ciblées ; rejeu complet de l’extension priorities. Les scripts d’audit et sorties structurées sont fournis.

La date de coupure des sources est le 9 octobre 2026 à 10:24:21 UTC. La date du présent audit n’étend pas cette fenêtre.

Installation et temps historique

F01 · P1 · Installation incomplète : 24 tests échouent dans un environnement neuf

Preuve : Reproduit. Avec Python 3.12.14 et les six dépendances exactes de requirements.txt, la suite donne 24 échecs et 113 succès. Les échecs observés viennent de numpy absent lors de la création de la fonction DuckDB precision_radius. Les chemins utilisant fetchdf() nécessitent aussi pandas ; la restitution de timestamps avec fuseau dépend de pytz dans cet environnement.

Conséquence : Un clone et la commande d'installation documentée ne suffisent pas. Un environnement personnel déjà enrichi masque facilement ce problème ; une réinstallation peut donc casser un projet qui fonctionnait auparavant.

Correction minimale : Déclarer numpy, pandas et pytz, ou retirer réellement les usages qui les nécessitent. Documenter la version Python testée. Les versions effectivement utilisées pour cet audit sont numpy 2.5.4, pandas 3.0.6 et pytz 2026.5 ; ce sont des versions vérifiées ici, pas une obligation de mise à jour globale.

Vérification ciblée : Après ajout de ces dépendances, les 137 tests passent. La dernière extension se régénère également à l'identique.

Code figé : requirements.txt:1-6 ; secfragility/database.py:87-109 ; secfragility/targeted_priorities.py:60-62

F02 · P1 · Le fuseau horaire peut changer un résultat historique

Preuve : Reproduit sur un cas minimal. quarter_series.prepare convertit knowledge_date, une DATE, en TIMESTAMPTZ sans fixer le fuseau de la connexion. database.create laisse le fuseau implicite. Dans une fixture contenant deux dépôts, les mêmes faits donnent un revenu as_known de 1 000 000 en UTC et 2 000 000 en America/New_York : le second dépôt devient admissible parce que la coupure absolue a changé.

Conséquence : Changer de machine ou de configuration horaire peut changer la photographie historique. Les 137 tests passent pourtant dans les deux environnements : le test existant vérifie surtout la date civile et ne couvre pas ce cas de frontière.

Correction minimale : Choisir explicitement la convention de publication et la fixer dans chaque connexion de calcul et de validation. Si la convention attendue est UTC, exécuter SET TimeZone='UTC' dans create et les autres connecteurs directs. Si une date SEC locale est voulue, la convertir explicitement dans cette zone, puis stocker l'instant UTC.

Vérification ciblée : Conserver un test de frontière avec deux publications le même jour et comparer l'instant de coupure ainsi que la valeur sous UTC, Europe/Paris et America/New_York.

Code figé : secfragility/database.py:87-94 ; secfragility/quarter_series.py:31-37 ; tests/test_publication_snapshot.py:10-39

Même fixture	Coupure en UTC	Revenu as_known
UTC	2025-02-05 00:00	1 000 000
Europe/Paris	2025-02-04 23:00	1 000 000
America/New_York	2025-02-05 05:00	2 000 000

Validation et conservation du dernier résultat

F03 · P1 · Les extensions échappent à la validation globale finale

Preuve : Anomalie présente dans les tables. phase3.finish appelle verify_delivery avant les extensions SoftBank, Dell, SMCI et priorities. L'extension ciblée réutilise une entité existante par son nom sans vérifier son statut. Deux arêtes monétaires aboutissent ainsi à CoreWeave Financing DDTL V, LLC, qui n'a aucune version confirmed dans entities.

Conséquence : Le résultat final viole no_pending_amount_edges, une règle que le projet impose lui-même. Les montants concernés sont 806 571 USD, côté BlackRock, et 5 319 000 USD, côté Ares, au 30 juin 2026. Leur valeur source n'est pas mise en cause par ce constat ; c'est la qualification de leur destinataire qui reste incohérente.

Correction minimale : Ajouter un contrôle global après toutes les extensions, juste avant la publication définitive. Dans entity(), traiter explicitement un nom déjà connu mais pending : confirmer l'identité juridique avec une preuve datée, sans inventer une consolidation au groupe, ou conserver l'observation sans arête monétaire admissible.

Vérification ciblée : La requête exacte du contrôle no_pending_amount_edges doit renvoyer zéro sur les huit tables finales. Vérifier aussi que les cas existants et les nouvelles entités passent par le même garde-fou.

Code figé : secfragility/phase3.py:118-149 ; secfragility/targeted_priorities.py:134-146 ; secfragility/targeted_priorities.py:190-202 ; secfragility/verify_delivery.py:15

F04 · P1 · Une interruption peut laisser des tables de générations différentes

Preuve : Reproduit par injection d'une erreur locale. database.export écrit les huit nouveaux fichiers dans un répertoire temporaire, puis remplace les anciens un par un. Une erreur simulée au remplacement de entities.parquet laisse documents, facts et observations nouveaux, avec les cinq autres tables anciennes. De plus, rebuild publie des tables intermédiaires dans tables/ avant la fin des calculs.

Conséquence : Un arrêt, une erreur disque ou un échec tardif d'extension peut dégrader le dernier jeu exploitable. Une transaction DuckDB ne rend pas atomique cette série de remplacements de fichiers. Le statut validated_ready_for_final_commit est aussi écrit avant la fin des extensions.

Correction minimale : Calculer et vérifier dans un nouveau dossier de résultats. Garder le dernier dossier validé intact jusqu'à la fin, puis changer un petit manifeste current.json qui indique le jeu à lire. Pour un usage manuel, un dossier daté validé et une étape explicite de sélection suffisent. Écrire le statut completed seulement à la toute fin.

Vérification ciblée : Une erreur volontaire au milieu du calcul ou de l'export doit laisser le jeu précédent entièrement lisible et désigné comme courant.

Code figé : secfragility/database.py:169-182 ; secfragility/rebuild.py:171-208 ; secfragility/phase3.py:118-148

REPRISE ET ATTRIBUTION

Deux garde-fous qui acceptent trop

F05 · P2 · La reprise d’un checkpoint ignore les modifications de code

Preuve : Reproduit. assembly_checkpoint.inputs calcule les empreintes du schéma, de la configuration, des requêtes et de certaines décisions. Il ne prend pas en compte secfragility/*.py. Modifier le code de calcul laisse donc inchangée la signature utilisée par resume pour accepter le checkpoint.

Conséquence : Après une correction, --resume peut réutiliser des résultats calculés avec l’ancien algorithme et exécuter la suite avec le nouveau. Ce mélange est difficile à voir et peut donner l’impression que la correction n’a aucun effet.

Correction minimale : Ajouter les empreintes des modules de calcul et des entrées effectivement consommées par l’étape sauvegardée. Une solution simple pour ce dépôt est un hash déterministe de secfragility/*.py, des fichiers SQL et de la configuration, plus les versions des dépendances de calcul.

Vérification ciblée : Le changement d’un module qui influence le calcul doit refuser la reprise et demander une reconstruction. La modification d’une simple note documentaire ne doit pas obliger à tout recalculer.

Code figé : secfragility/assembly_checkpoint.py:7-31

F06 · P2 · La présence du nom dans le bloc ne prouve pas son lien avec le montant

Preuve : Contre-exemple accepté par le validateur. validate_semantics exige que la citation soit présente dans le bloc et que la contrepartie nommée figure quelque part dans ce bloc. Ces conditions sont indépendantes. Le bloc « We paid OpenAI $100. We paid Anthropic $200. » accepte une observation citant la première phrase, montant 100 USD, contrepartie Anthropic.

Conséquence : Une erreur d’annotation humaine peut passer tous ces contrôles de présence et affecter le graphe. La sélection ciblée utilise parfois le document entier comme bloc, ce qui élargit la surface de cette confusion. Aucune attribution fausse déjà publiée n’est démontrée par cette fixture.

Correction minimale : Pour une attribution directe, exiger la contrepartie dans la citation exacte du montant et vérifier la ligne ou le contexte choisi. Pour une attribution par renvoi documentaire, imposer une preuve de liaison séparée et explicite. Éviter de faire porter au seul nom trouvé dans un grand bloc une garantie sémantique qu’il ne fournit pas.

Vérification ciblée : Le contre-exemple fourni doit être rejeté. Les renvois documentaires légitimes doivent rester possibles avec leur preuve distincte.

Code figé : secfragility/observations.py:15-56 ; secfragility/targeted_priorities.py:95-116

Pouvoir retrouver les mêmes données

F07 · P2 · Neuf sources de métadonnées ne sont pas conservées dans le dépôt audité

Preuve : Inventaire et empreintes vérifiés. Sur 12 197 documents référencés, 12 182 sont retrouvés au chemin attendu dans le clone ou les six archives. Six autres sont retrouvés sous un autre chemin avec le même SHA-256. Neuf réponses submissions SEC restent introuvables, même par recherche de contenu identique dans les archives et les fichiers suivis.

Conséquence : 52 lignes de mesures de signaux référencent ces neuf documents. Cela n'établit pas que ces signaux sont faux, mais empêche de vérifier leur source brute entièrement hors ligne à partir de ce clone. metadata_signals.run lit directement les chemins de cache attendus ; les six sources sous un autre chemin nécessitent également un remappage.

Correction minimale : Conserver les neuf réponses exactes depuis le cache d'origine si elles sont encore disponibles, puis les ajouter à une sauvegarde avec manifeste. Rétablir les six chemins équivalents. Un téléchargement actuel ne doit pas être présenté comme la même photographie historique sans contrôle du hash.

Vérification ciblée : Un précontrôle des documents doit retrouver le contenu brut de chaque document par son empreinte avant la reconstruction. La liste complète des neuf URLs est dans l'annexe de preuves.

Code figé : secfragility/metadata_signals.py:8-25 ; secfragility/deepening_backup.py:9-35 ; README.md:19

F08 · P2 · Le dossier audit/ et son protocole ne décrivent pas le résultat courant complet

Preuve : Écart constaté entre instantanés. audit/measures.parquet contient 552 039 lignes contre 599 037 dans tables/ ; audit/controls.parquet en contient 298 632 contre 340 172. Dell, SMCI et SoftBank sont absents de ces deux tables historiques. Des compléments séparés existent, mais AUDITOR.md annonce encore le périmètre initial de onze groupes ; README conserve aussi un intitulé « modèle courant » à vingt-quatre groupes.

Conséquence : En revenant au projet plus tard, on peut auditer ou lire le mauvais instantané et croire avoir couvert les vingt-six groupes actuels. Les anciens fichiers sont utiles pour conserver l'historique ; leur présence n'est pas une perte de données.

Correction minimale : Marquer clairement le dossier historique et donner un seul point d'entrée pour le résultat actuel : commit, vingt-six groupes, tables courantes, date de coupure et liste des compléments. Actualiser les indications « courant » et le protocole d'audit, sans écraser les anciens états.

Vérification ciblée : Depuis le README, un lecteur doit pouvoir identifier sans deviner les huit fichiers actuels et le périmètre exact de chaque dossier d'audit.

Code figé : audit/AUDITOR.md:1-5 ; README.md:1-30

Relier la preuve au résultat réellement produit

F09 · P2 · Les preuves de tests ne sont pas rattachées au code réellement empaqueté

Preuve : Défaut établi par lecture du code. targeted_package lit des booléens enregistrés dans replay_verification.json, puis calcule les empreintes du code actuel. Il ne compare pas ces empreintes à celles du code réellement testé. Les nombres full_tests_passed=137 et isolated_tests_passed=5 sont écrits en dur. Comparer le contenu du ZIP au code courant prouve la copie, pas l'exécution antérieure des tests sur ce code.

Conséquence : Après une modification locale, un ancien fichier de preuve peut encore autoriser un paquet qui se présente comme testé. Le rejeu indépendant réalisé pendant cet audit confirme bien l'état actuel ; le défaut concerne la fiabilité de la prochaine attestation.

Correction minimale : Enregistrer les empreintes des fichiers et le résultat réel de la commande au moment du test ou du rejeu. Refuser l'empaquetage si le code ou les entrées ont changé. Remplacer les compteurs constants par les valeurs de cette exécution.

Vérification ciblée : Changer un module après le rejeu doit invalider la preuve jusqu'à une nouvelle vérification. Il n'est pas nécessaire d'ajouter un service externe pour cela.

Code figé : secfragility/targeted_package.py:22-27 ; secfragility/targeted_package.py:34-50

F10 · P3 · 255 lignes de ratios portent une devise monétaire

Preuve : Incohérence de métadonnées présente. 251 lignes SoftBank et quatre lignes CRWV ont unit=ratio avec currency non nulle. Plusieurs appels SoftBank passent explicitement currency=JPY pour des ratios ; le ratio de liquidité borné CRWV conserve USD. La règle est déjà correctement appliquée aux ratios de la dernière extension.

Conséquence : La valeur du ratio n'est pas rendue fausse par cette étiquette. En revanche, un export, un formatteur ou un futur calcul générique peut le traiter comme un montant monétaire. Le schéma devient moins uniforme.

Correction minimale : Mettre currency à NULL pour les résultats sans dimension. Garder la devise des termes dans les faits de provenance, où elle reste nécessaire pour vérifier leur compatibilité. Appliquer la même règle aux ratios bornés et aux lignes sans valeur.

Vérification ciblée : La requête unit='ratio' AND currency IS NOT NULL doit renvoyer zéro après harmonisation.

Code figé : secfragility/softbank_integration.py:277-310 ; secfragility/softbank_integration.py:488 ; secfragility/deepening_assembly.py:202-223

UTILISATION LOCALE

Chemins, sessions et effets de bord

F11 · P3 · Certains chemins empêchent une utilisation locale portable

Preuve : Constaté dans les points d'entrée. Les paquetages SoftBank et Dell utilisent un chemin absolu propre à l'ancien environnement /codex/browser/projectless/.../output. Leur interface ne permet pas de choisir la destination. targeted_package propose déjà --output. Les vues ajoutées à queries.sql ouvrent aussi des Parquet relatifs au répertoire courant, même lorsqu'une fonction reçoit un autre root.

Conséquence : Des commandes peuvent échouer sur un PC ordinaire ou lire des données d'un autre dossier lorsque le répertoire de lancement change. Les tests réussis depuis la racine du clone ne prouvent pas cette portabilité.

Correction minimale : Utiliser un dossier output/ relatif au projet, et un argument --output pour les paquetages. Résoudre les chemins de lecture à partir du root transmis. Charger les vues propres aux extensions quand leurs sources sont disponibles.

Vérification ciblée : Le même point d'entrée doit fonctionner depuis la racine et avec un root explicite depuis un autre répertoire. La génération locale d'un paquet doit pouvoir choisir un dossier accessible.

Code figé : secfragility/dell_package.py:7 ; secfragility/softbank_package.py:6 ; secfragility/targeted_package.py:56 ; queries.sql:62-86

F12 · P3 · Le verrou réseau est contrôlé après l'envoi de la requête

Preuve : Reproduit avec un transport simulé. SecClient.get appelle le transport puis _journal, qui appelle enfin lock.touch. Si le verrou appartient déjà à une autre session, une requête est tout de même envoyée avant l'erreur. Le contre-exemple fourni compte un appel simulé avant « Le verrou de cette session a été perdu ».

Conséquence : Le cas se produit surtout lors d'une session devenue obsolète ou de deux lancements locaux. Le verrou ne garantit alors plus strictement qu'un seul processus accède au réseau. Aucun dépassement n'a été trouvé dans le journal historique contrôlé.

Correction minimale : Vérifier et rafraîchir le verrou juste avant chaque appel au transport, y compris après une attente et avant une nouvelle tentative. La correction est locale au client existant.

Vérification ciblée : Avec un jeton de verrou incorrect, le transport simulé doit compter zéro appel. Conserver le contrôle du verrou au moment du journal également.

Code figé : secfragility/network.py:70-75 ; secfragility/network.py:101-108 ; secfragility/network.py:140-157

Comportements locaux à connaître

database.create impose 8 Go de mémoire DuckDB et quatre threads. Cela mérite simplement un réglage dans config.yaml si ta machine est plus petite. Le dossier de spill est commun au root : une seule reconstruction à la fois reste l'usage le plus simple.

Le checkpoint de lecture exécute git add -A et peut faire git push lorsque work/github_backup.json contient enabled=true, ce qui est le cas de l'état audité. Cette sauvegarde est intentionnelle dans le projet. Pour travailler exclusivement en local, mettre ce booléen à false suffit à supprimer le push ; le commit local reste un autre effet de bord à connaître. Aucun de ces points d'entrée n'a été exécuté pendant cet audit.

Code figé : secfragility/database.py:87-94 ; secfragility/checkpoint.py:40-77 ; work/github_backup.json:1-7

Résultats sur les huit tables actuelles

Table	Lignes	Lecture
documents	12 197	Pièces et métadonnées sources
facts	1 435 879	Occurrences de faits, pas autant de chiffres uniques
observations	17 958	Lectures, qualifications et abstentions
entities	775	Identités et versions d’appartenance
links	927	Relations et arêtes monétaires
measures	599 037	Mesures, conclusions, événements et états ND
controls	340 172	Tests comptables et de cohérence
exclusions	16 793	Limites, conflits et exclusions explicites

2 423 738 lignes au total. Les contrôles portent sur les tables complètes, pas seulement sur un échantillon de leurs premières lignes.

Invariants vérifiés sans anomalie dans le périmètre de ces requêtes

Aucune valeur numérique sur une mesure ND ; aucun motif ND manquant ; aucune borne inversée ou incomplète ; aucun fait nil doté d’une valeur ; aucun zéro explicite incohérent ; aucune abstention avec montant ; aucun motif de contrôle manquant ; aucune arête de financement monétaire au seul stade signé.

Aucun fait ou observation orphelin de document ; aucun tagged_fact_id manquant ; aucune observation référencée par une arête introuvable. Toutes les références de provenance des mesures numériques examinées se résolvent ; aucune dépendance de niveau E/F n’a été trouvée. Les montants et périodes des observations tagged_reference concordent avec les faits associés.

Avec une convention UTC explicite, aucune date future ni période inversée n’a été trouvée dans les requêtes exécutées. Cette cohérence de l’instantané actuel n’annule pas le défaut de fuseau reproduit au recalcul.

4 858 ratios disposant simultanément d’une valeur, d’un numérateur et d’un dénominateur non nul concordent avec numérateur / dénominateur à 10^{-6} près. Ce contrôle vérifie l’arithmétique publiée ; il ne certifie pas à lui seul le choix comptable des termes.

INTERPRÉTATION DES RÉSULTATS

Ce que les statuts permettent de conclure

Statut des mesures	Nombre	Avec value non nulle
not_determinable	504 066	0
computed	88 869	29 683
partial	6 098	5 061
bounded	4	0 ; bornes présentes

Environ 84,1 % des lignes de mesures sont ND. Ce taux vient aussi du grand univers de cellules attendues et des lacunes explicites des publications. Il ne constitue pas un taux d’erreur. Inversement, computed ne signifie pas toujours « montant disponible » : le modèle contient aussi des conclusions descriptives. Au total, 34 744 lignes ont une valeur, auxquelles s’ajoutent quatre résultats bornés.

Statut des contrôles	Nombre	Sens
ok	335 554	Contrôle satisfait selon sa définition
mismatch	219	Écart conservé et expliqué
not_testable	4 382	Termes ou preuve insuffisants
tautological	17	Égalité non indépendante

320 960 des contrôles ok proviennent de c4_restatement_detection. Un taux global de succès écrase donc les contrôles comptables plus rares. Pour lire le résultat, regarder les familles de contrôles et les périodes concernées.

Les 219 écarts se répartissent principalement en 67 contrôles EPS, 64 contrôles de taux d’impôt, 50 contrôles de bilan et 19 contrôles de flux. Le groupe external:0001858681 concentre notamment 48 écarts de bilan, 34 EPS et 15 flux. Ces chiffres comptent des lignes de vues ou périodes, pas autant d’événements économiques distincts.

Le programme conserve de bonnes distinctions : zéro contre absence ; capacité signée contre tirage ; nominal, coût et juste valeur ; événement contractuel contre versement ; IFRS contre US GAAP ; vue as_known contre vue revised. Les faits bruts et les décisions de qualification sont séparés. Les contreparties anonymes ne deviennent pas automatiquement nommées et les limites de divulgation restent visibles.

Les deux arêtes de F03 et les 255 étiquettes de devise de F10 sont les exceptions structurelles confirmées. Aucun total global d’exposition ne doit être déduit en additionnant les lignes du graphe : les bases, dates et positions peuvent se recouvrir.

VÉRIFICATION INDÉPENDANTE

Tests, rejeu et intégrité des sources

Exécution ou contrôle	Résultat observé
Python 3.12.14 + requirements.txt	24 failed, 113 passed, 2,80 s
Après numpy, pandas, pytz ; UTC	137 passed, 13,35 s avec couverture
Même suite ; America/New_York	137 passed, 8,46 s
Régénération des sélections priorities	56 observations identiques ; 51,47 s
Application de priorities dans un dossier neuf	8 fichiers Parquet identiques ; 94,98 s
Tests ciblés dans ce dossier neuf	5 passed, 1,05 s
Empreintes des objets LFS	169 / 169 conformes
Empreintes des six archives de sources	6 / 6 conformes

Le rejeu est parti du snapshot antérieur conservé et des originaux de la dernière extension. Il a réexécuté author_notes.py, validé les citations et les montants inline, puis appliqué l'extension. Les SHA-256 des huit sorties égalent ceux des tables courantes. Cette preuve est plus forte qu'une comparaison de quelques compteurs ; elle reste limitée à cette extension.

Couverture de la suite de 137 tests : 1 690 instructions couvertes sur 7 329, soit 23,06 % ; 592 branches sur 2 970, soit 19,93 %. Quarante-six modules non vides ne sont pas exécutés par cette suite. Cette mesure précède le rejeu indépendant, qui n'a pas été inclus dans le taux de couverture.

Les tests sont utiles sur les arrondis, les différences de trimestres, les changements de devise et de périmètre, les citations et les cas métier sélectionnés. Une partie importante lit toutefois les fichiers déjà calculés. C'est pourquoi une suite verte n'attrape pas les défauts d'export, de reprise, de fuseau et de validation après extension.

Journal historique : 10 630 requêtes enregistrées ; maximum observé de cinq requêtes dans une seconde glissante ; un HTTP 403 suivi d'une attente de 600,86 secondes avant la requête suivante. Le contrôle est fondé sur le journal conservé. Aucune nouvelle requête SEC n'a été envoyée pour cet audit.

L'analyse de dépendances remonte une seule vulnérabilité distincte pour lxml 6.0.2, dupliquée dans la réponse du scanner : GHSA-vfmq-68hx-4jfw, corrigée en 6.1.0. Les chemins iterparse/ETCompatXMLParser visés ne sont pas utilisés dans secfragility ; aucun chemin exploitable n'a été démontré. Ce point n'est pas classé parmi les bugs prioritaires.

PROVENANCE RESTANTE

Les neuf fichiers absents et la limite du rejeu global

Groupe / identifiant	Réponse submissions absente
external:0001859392	CIK0001859392.json
external:0001819989	CIK0001819989.json
external:0001878848	CIK0001878848.json
external:0001083301	CIK0001083301-submissions-001.json
external:0001839341	CIK0001839341.json
DELL	CIK0001571996-submissions-001.json
external:0001462056	CIK0001462056.json
external:0001513845	CIK0001513845.json
DELL	CIK0001571996.json

Les neuf documents sont des réponses JSON de métadonnées. Leurs SHA attendus et leurs chemins exacts sont conservés dans alternative-source-locations.json. Aucune pièce financière principale n’a été signalée manquante par cet inventaire final. Les six autres divergences de chemin disposent d’un contenu identique retrouvé dans les archives.

Les 52 lignes dépendantes concernent sig_distress_8k_items, sig_late_filing et sig_auditor_change_or_nonreliance sur quatre identifiants de groupe. Les vues as_known et revised peuvent représenter le même signal : ces 52 lignes ne sont pas 52 événements uniques.

Ce qui reste non certifié

La reconstruction intégrale de toutes les étapes depuis les sources brutes n’a pas été exécutée. Le dernier overlay a été rejoué, les sources conservées ont été vérifiées et les tables finales ont été contrôlées ; ces trois opérations ne remplacent pas une reconstruction historique complète.

L’audit ne certifie pas la complétude de tout EDGAR ni la justesse sémantique de chaque choix de concept, de chaque attribution ou de chacune des 34 744 valeurs. Il ne prétend pas avoir relu humainement chaque page des 12 197 documents. La vérification des empreintes garantit l’identité avec les références conservées dans le dépôt ; elle ne constitue pas un nouveau téléchargement authentifiant chaque dépôt public.

Le protocole financier autonome de audit/AUDITOR.md, qui demande notamment une nouvelle dérivation de trente cellules depuis EDGAR et un inventaire externe des dépôts, est distinct de l’audit du dépôt demandé ici. Il n’a pas été présenté comme intégralement exécuté. Les limites de divulgation déjà reconnues par le projet sont conservées comme limites, sans être comptées artificiellement comme des bugs.

CORRECTIONS PROPORTIONNÉES

Un ordre de travail simple

Ordre	Modification	Critère de fin
1	Déclarer les dépendances et la version Python testée.	Le clone neuf passe les 137 tests avec la seule installation documentée.
2	Fixer les fuseaux et la convention des dates de publication.	La fixture de frontière produit les mêmes instant et valeur dans les trois zones.
3	Valider les huit tables après tous les ajouts ; traiter DDTL V.	Zéro arête monétaire vers une entité sans confirmation documentaire.
4	Conserver le dernier résultat validé jusqu'à la fin du suivant.	Un échec à mi-parcours ne change pas le jeu courant.
5	Lier les checkpoints et preuves de test aux empreintes du code.	Une modification de calcul invalide les reprises et attestations concernées.
6	Renforcer le lien citation / contrepartie / montant.	Le contre-exemple d'attribution est rejeté ; les renvois explicites restent possibles.
7	Récupérer les métadonnées et clarifier le résultat courant.	Sources brutes retrouvables et une seule indication non ambiguë des tables à utiliser.
8	Harmoniser les ratios, chemins et contrôle du verrou.	Les trois incohérences locales restantes sont supprimées.

Ces changements peuvent être faits dans les modules actuels. Les vérifications à ajouter sont les quelques scénarios qui ont permis de reproduire les défauts ; un objectif abstrait de couverture n'est pas nécessaire pour terminer ces corrections.

Comment exploiter les preuves jointes

L'archive de preuves contient les résultats lisibles en JSON, les journaux de tests pertinents et les scripts d'audit. constats_audit.json associe chaque constat à ses fichiers et preuves. LISEZ_MOI.txt précise les commandes, le périmètre de chaque script et la révision attendue.

Les preuves lourdes du dépôt ne sont pas dupliquées : les scripts lisent ton clone au commit audité. replay_targeted.py crée un nouveau répertoire de travail et préserve les Parquet d'entrée ; les scripts de contrôle ne modifient pas le contenu suivi du dépôt.

Références externes utilisées pour interpréter les outils

DuckDB - Timestamp Issues : effets du fuseau sur les conversions DATE / TIMESTAMPTZ. La preuve de F02 provient toutefois de l'exécution locale reproduite.

lxml - avis officiel GHSA-vfmq-68hx-4jfw : versions concernées, parseurs visés et correctif.

Toutes les références de code du rapport pointent sur le commit figé, et non sur une branche susceptible de changer. L'empreinte complète est reprise dans constats_audit.json et dans les preuves de comparaison.